



4.7 while Repetition Statement

- ▶ A **repetition statement** (also called a **looping statement** or a **loop**) allows you to specify that a program should repeat an action while some condition remains true.

*While there are more items on my shopping list
Purchase next item and cross it off my list*

- ▶ “There are more items on my shopping list” is true or false.
 - If true, “Purchase next item and cross it off my list” is performed.
 - Performed repeatedly while the condition remains true.
 - The statement contained in the *While* repetition statement constitutes the body of the *While*, which can be a single statement or a block.
 - Eventually, the condition will become false, the repetition will terminate, and the first pseudocode statement after the repetition statement will execute.



4.7 while Repetition Statement (cont.)

- ▶ Consider a program segment designed to find the first power of 3 larger than 100. Suppose the integer variable `product` has been initialized to 3.
- ▶ When the following `while` repetition statement finishes executing, `product` contains the result:

- `int product = 3;`

```
while ( product <= 100 )  
    product = 3 * product;
```



Common Programming Error 4.2

A logic error called an **infinite loop**, in which the repetition statement never terminates, occurs if you do not provide an action in a `while` statement's body that eventually causes the condition in the `while` to become false normally. This can make a program appear to “hang” or “freeze” if the loop body does not contain statements that interact with the user.

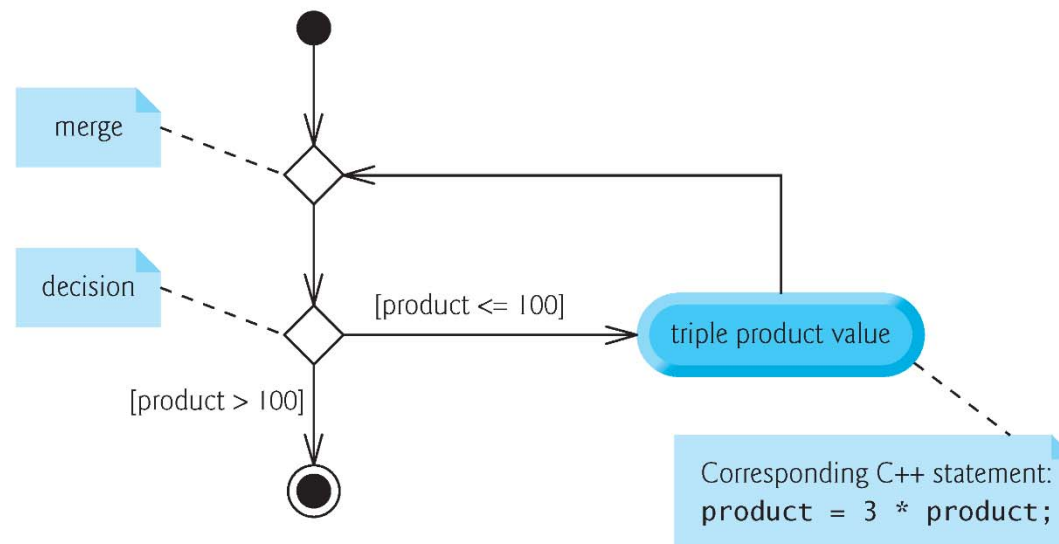


Fig. 4.6 | while repetition statement UML activity diagram.



4.10 Formulating Algorithms: Nested Control Statements

- ▶ Consider the following problem statement:
 - A college offers a course that prepares students for the state licensing exam for real estate brokers. Last year, ten of the students who completed this course took the exam. The college wants to know how well its students did on the exam. **You've been asked to write a program to summarize the results. You've been given a list of these 10 students. Next to each name is written a 1 if the student passed the exam or a 2 if the student failed.**

Your program should analyze the results of the exam as follows:

- a) Input each test result (i.e., a 1 or a 2). Display the prompting message "Enter result" each time the program requests another test result.
- b) Count the number of test results of each type.
- c) Display a summary of the test results indicating the number of students who passed and the number who failed.
- d) If more than eight students passed the exam, print the message "**Bonus to instructor!**"



4.10 Formulating Algorithms: Nested Control Statements (cont.)

- ▶ After reading the problem statement carefully, we make the following observations:
 - Must process test results for 10 students. A counter-controlled loop can be used because the number of test results is known in advance.
 - Each test result is a number—either a 1 or a 2. Each time the program reads a test result, the program must determine whether the number is a 1 or a 2. For simplicity, we test only for a 1 in our algorithm. If the number is not a 1, we assume that it's a 2.
 - Two counters keep track of the exam results—one to count the number of students who passed and one to count the number of students who failed.
 - After the program has processed all the results, it must decide whether more than eight students passed the exam.



4.10 Formulating Algorithms: Nested Control Statements (cont.)

- ▶ Pseudocode representation of the top:

Analyze exam results and decide whether tuition should be raised

- ▶ Once again, it's important to emphasize that the top is a *complete* representation of the program, but several refinements are likely to be needed before the pseudocode evolves naturally into a C++ program.

- ▶ Our first refinement is

Initialize variables

Input the 10 exam results, and count passes and failures

Print a summary of the exam results and decide if tuition should be raised

- ▶ Further refinement is necessary.



4.10 Formulating Algorithms: Nested Control Statements (cont.)

- ▶ Counters are needed to record the passes and failures, a counter will be used to control the looping process and a variable is needed to store the user input.
- ▶ The pseudocode statement

Initialize variables

can be refined as follows:

Initialize passes to zero

Initialize failures to zero

Initialize student counter to one



4.10 Formulating Algorithms: Nested Control Statements (cont.)

- ▶ The following pseudocode statement requires a loop that successively inputs the result of each exam

Input the 10 exam results, and count passes and failures

- ▶ 10 exam results, so counter-controlled looping is appropriate.
- ▶ Nested inside the loop, an **if...else** statement will determine whether each exam result is a pass or a failure and will increment the appropriate counter.
- ▶ The refinement of the preceding pseudocode statement is then

While student counter is less than or equal to 10

Prompt the user to enter the next exam result

Input the next exam result

If the student passed

Add one to passes

Else

Add one to failures

Add one to student counter



4.10 Formulating Algorithms: Nested Control Statements (cont.)

- ▶ The pseudocode statement

Print a summary of the exam results and decide whether tuition should be raised

can be refined as follows:

Print the number of passes

Print the number of failures

If more than eight students passed

Print “Bonus to instructor!”

- ▶ The complete second refinement appears in Fig. 4.15.



```
1  Initialize passes to zero
2  Initialize failures to zero
3  Initialize student counter to one
4
5  While student counter is less than or equal to 10
6      Prompt the user to enter the next exam result
7      Input the next exam result
8
9      If the student passed
10         Add one to passes
11     Else
12         Add one to failures
13
14     Add one to student counter
15
16 Display the number of passes
17 Display the number of failures
18
19 If more than eight students passed
20     Display "Bonus to instructor!"
```

Fig. 4.15 | Pseudocode for examination-results problem.



```
1 // Fig. 4.16: fig04_16.cpp
2 // Examination-results problem: Nested control statements.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     // initializing variables in declarations
9     unsigned int passes = 0; // number of passes
10    unsigned int failures = 0; // number of failures
11    unsigned int studentCounter = 1; // student counter
12
```

Fig. 4.16 | Examination-results problem: Nested control statements. (Part 1 of 4.)



```
13 // process 10 students using counter-controlled loop
14 while ( studentCounter <= 10 )
15 {
16     // prompt user for input and obtain value from user
17     cout << "Enter result (1 = pass, 2 = fail): ";
18     int result = 0; // one exam result (1 = pass, 2 = fail)
19     cin >> result; // input result
20
21     // if...else nested in while
22     if ( result == 1 )           // if result is 1,
23         passes = passes + 1;     // increment passes;
24     else                         // else result is not 1, so
25         failures = failures + 1; // increment failures
26
27     // increment studentCounter so loop eventually terminates
28     studentCounter = studentCounter + 1;
29 } // end while
30
31 // termination phase; display number of passes and failures
32 cout << "Passed " << passes << "\nFailed " << failures << endl;
33
34 // determine whether more than eight students passed
35 if ( passes > 8 )
36     cout << "Bonus to instructor!" << endl;
37 } // end main
```

Fig. 4.16 | Examination-results problem: Nested control statements. (Part 2 of 4.)



```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Passed 6
Failed 4
```

Fig. 4.16 | Examination-results problem: Nested control statements. (Part 3 of 4.)



```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Passed 9
Failed 1
Bonus to instructor!
```

Fig. 4.16 | Examination-results problem: Nested control statements. (Part 4 of 4.)



4.10 Formulating Algorithms: Nested Control Statements (cont.)

- ▶ The `if...else` statement (lines 22–25) for processing each result is nested in the `while` statement.
- ▶ The `if` statement in lines 35–36 determines whether more than eight students passed the exam and, if so, outputs the message "**Bonus to instructor!**".



4.11 Assignment Operators

- ▶ C++ provides several **assignment operators** for abbreviating assignment expressions.
- ▶ The `+=` operator adds the value of the expression on the right of the operator to the value of the variable on the left of the operator and stores the result in the variable on the left of the operator.
- ▶ Any statement of the form
 - *variable = variable operator expression;*
- ▶ in which the same *variable appears on both sides of the assignment operator and operator is one of the binary operators `+`, `-`, `*`, `/`, or `%` (or others we'll discuss later in the text), can be written in the form
 - *variable operator= expression;**
- ▶ Thus the assignment `c += 3` adds 3 to c.
- ▶ Figure 4.17 shows the arithmetic assignment operators, sample expressions using these operators and explanations.



Assignment operator	Sample expression	Explanation	Assigns
<i>Assume: int c = 3, d = 5, e = 4, f = 6, g = 12;</i>			
+=	c += 7	c = c + 7	10 to c
-=	d -= 4	d = d - 4	1 to d
*=	e *= 5	e = e * 5	20 to e
/=	f /= 3	f = f / 3	2 to f
%=	g %= 9	g = g % 9	3 to g

Fig. 4.17 | Arithmetic assignment operators.



4.12 Increment and Decrement Operators

- ▶ C++ also provides two unary operators for adding 1 to or subtracting 1 from the value of a numeric variable.
- ▶ These are the unary **increment operator**, ++, and the unary **decrement operator**, --, which are summarized in Fig. 4.18.



Operator	Called	Sample expression	Explanation
++	preincrement	++a	Increment a by 1, then use the new value of a in the expression in which a resides.
++	postincrement	a++	Use the current value of a in the expression in which a resides, then increment a by 1.
--	predecrement	--b	Decrement b by 1, then use the new value of b in the expression in which b resides.
--	postdecrement	b--	Use the current value of b in the expression in which b resides, then decrement b by 1.

Fig. 4.18 | Increment and decrement operators.



```
1 // Fig. 4.19: fig04_19.cpp
2 // Preincrementing and postincrementing.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     // demonstrate postincrement
9     int c = 5; // assign 5 to c
10    cout << c << endl; // print 5
11    cout << c++ << endl; // print 5 then postincrement
12    cout << c << endl; // print 6
13
14    cout << endl; // skip a line
15
16    // demonstrate preincrement
17    c = 5; // assign 5 to c
18    cout << c << endl; // print 5
19    cout << ++c << endl; // preincrement then print 6
20    cout << c << endl; // print 6
21 }
```

Fig. 4.19 | Preincrementing and postincrementing. (Part 1 of 2.)



5	
5	
6	
5	
6	
6	

Fig. 4.19 | Preincrementing and postincrementing. (Part 2 of 2.)